The Journal of Open Source Software

GECKO: A researcher-in-the-loop tool for gaze event classification in Kinarm robotic experiments

Luke Kroon [1], **Angus P. Muttee** [1], **Blake A. Hollinger** [1], **and Tarkeshwar Singh** [1,2¶]

1 Department of Kinesiology, The Pennsylvania State University, University Park, PA-16802, USA 2 Penn State Neuroscience Institute, The Pennsylvania State University, University Park, PA-16802, USA
¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 09 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

GECKO (Gaze Event Classification in the Kinarm, Open-access) is an open-source Python desktop application for visualizing, annotating, and exporting eye-tracking and limb-movement data recorded on Kinarm robots (Scott, 1999). In Kinarm experiments, participants reach toward visual stimuli projected into a horizontal plane at hand level, which complicates computing eye-movement kinematics and identifying gaze events: fixations (the eyes hold still), smooth pursuits (the eyes follow a moving object), and saccades (the eyes jump between points). GECKO reads native .kinarm files, cleans and fills missing gaze samples, computes the ocular kinematics described by Singh et al. (2016), and presents each trial as an interactive plot on which a researcher labels gaze events frame by frame. Labels are exported as per-frame codes alongside selected kinematic channels, with every processing decision recorded.

Statement of need

Kinarm exoskeleton and endpoint robots (Scott, 1999) measure upper-limb movement with high spatial and temporal precision and are increasingly paired with eye trackers to study eye–hand coordination during reaching and interception (Figure 1,A). The `.kinarm` data files store synchronized gaze and kinematic channels, but the eye tracker's built-in event flags were not designed for this geometry and cannot detect smooth pursuits (Singh et al., 2016). Two preprocessing steps are also needed before gaze events can be classified, manually or automatically.

First, gaze samples lost to blinks or tracking failures are recorded as large sentinel values, which must be detected, removed, and interpolated before the signal can be differentiated. Second, point-of-regard (POR) is recorded as (x, y) coordinates in the transverse plane (Figure 1,C). Because that plane lies in front of and below the eye, viewing distance varies across the workspace, so converting POR to angular kinematics is a three-dimensional geometric problem rather than the scalar calculation used for a frontoparallel display (Figure 1,B). GECKO performs both steps, transforming each POR sample into eye-centered spherical coordinates to compute gaze angular velocity, the quantity that distinguishes saccades, smooth pursuits, and fixations during labeling.

DRAFT

The Journal of Open Source Software

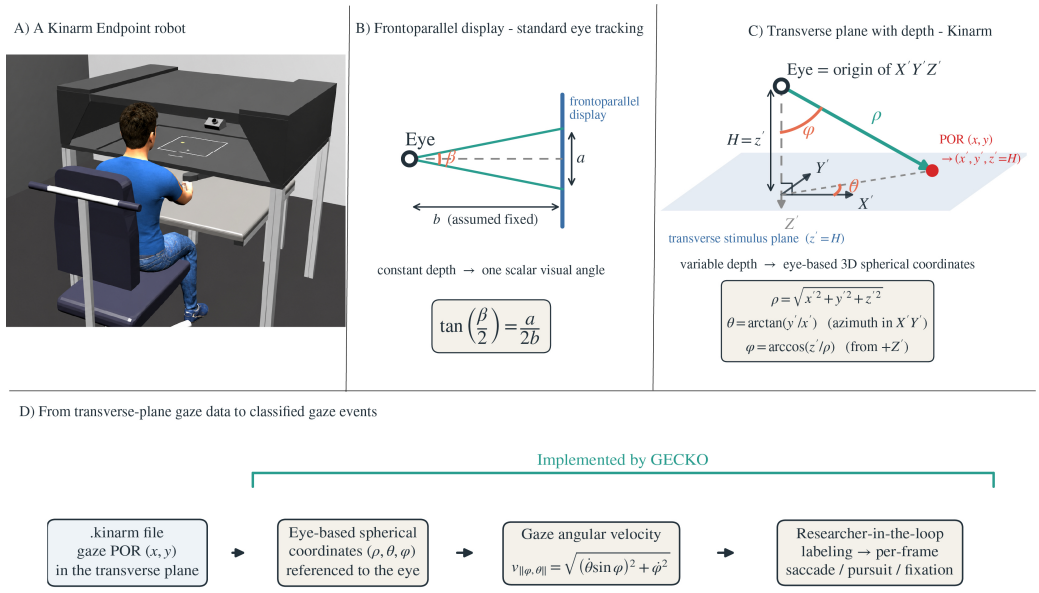


Figure 1: Gaze geometry in a Kinarm workspace. (A) A Kinarm Endpoint robot; targets are projected into a horizontal plane at hand level and viewed by a remote eye tracker. (B) Screen-based eye tracking assumes a fixed viewing distance b, so a stimulus of extent a subtends a single visual angle β. (C) In a Kinarm workspace viewing distance varies, so following Singh et al. (2016) each point-of-regard (POR) is transformed into eye-centered spherical coordinates (ρ, θ, φ), with the stimulus plane at eye height H. (D) GECKO implements this transform, computes gaze angular velocity, and supports frame-level labeling of saccades, pursuits, and fixations.

Singh et al. (2016) introduced a geometric method for computing ocular kinematics and classifying gaze events in robotic workspaces with static and moving targets, but no maintained software implementation exists, so laboratories reimplement it in ad hoc scripts. A second need concerns the labels themselves: manual classification by expert annotators is widely treated as the ground truth for evaluating event detection (Startsev & Zemblys, 2023), yet no tool supports producing such labels at scale for Kinarm data. GECKO addresses both needs. It reads `.kinarm` files directly and implements the Singh et al. (2016) spherical-coordinate transform, angular velocity, and foveal visual radius (FVR), the radius of the fovea's footprint on the stimulus plane, which grows with viewing distance and determines whether gaze is close enough to a target to count as fixating or pursuing it. These computations are surfaced in a labeling interface designed for imperfect gaze data (Figure 1,D). The target users are motor-control and rehabilitation researchers who record gaze alongside Kinarm limb kinematics and need a documented pipeline that requires no programming.

State of the field

A mature ecosystem of automatic eye-movement event-detection algorithms exists, including adaptive velocity-based detection (Nyström & Holmqvist, 2010), noise-robust fixation clustering (I2MC) (Hessels et al., 2017), and classification for dynamic stimuli (REMoDNaV) (Dar et al., 2021). These were developed for a participant viewing a two-dimensional display at approximately constant depth. None reads `.kinarm` files, accounts for the variable eye-to-stimulus distance of a transverse workspace, or displays gaze alongside the recorded motion of the targets, which is what separates smooth pursuit from fixation when stimuli move. Comparative evaluations also report substantial disagreement among algorithms and between algorithms and human coders (Andersson et al., 2017), so a new recording geometry needs reliable manual labels before any automatic method can be validated in it. Even in screen-based research, automatic detections are sometimes corrected by hand (Hessels et al., 2017).

DRAFT

GECKO is, to our knowledge, the first openly available tool that (i) reads the native `.kinarm` format, (ii) implements the Singh et al. (2016) geometric ocular-kinematics pipeline, and (iii) couples it to frame-level, human-in-the-loop labeling with full export control. We built a dedicated tool rather than extending an existing package because all three pieces would have had to be added, and because manual labeling is GECKO's core purpose rather than a correction step applied after automatic detection.

Software design

GECKO's architecture mirrors the hierarchical structure of Kinarm data itself: files contain trials, and trials contain channels. The loaded-file object maps onto trials, which map onto individual channels, and the GUI builds directly on this object model without flattening or restructuring the underlying data. This reduces the risk of cross-trial indexing errors and makes the codebase intuitive to extend, compared with the alternative of loading everything into flat data frames at startup. The core sentinel-cleaning, interpolation, and gaze-metric computations are decoupled from the interface, so these modules can be imported into standalone scripts for batch reprocessing and for sharing reproducible analysis code alongside publications. The pipeline is implemented in Python and builds on NumPy (Harris et al., 2020) for the array operations underlying the spherical-coordinate and angular-velocity computations, SciPy (Virtanen et al., 2020) for signal filtering (the Butterworth low-pass and Savitzky–Golay differentiation described below), and Matplotlib (Hunter, 2007) for the interactive trial plots that underpin the labeling interface.

All processing parameters are centralized in a single configuration file so that labs can adapt the tool to their own setup without modifying the pipeline. These include the sentinel threshold, the gap length below which interpolation happens automatically, the assumed eye height above the stimulus plane, the foveal cone angle used for FVR, and the filter settings: a 20 Hz fourth-order zero-phase Butterworth low-pass applied to gaze before metric computation, and a Savitzky–Golay filter used internally for the time derivatives in the angular-velocity calculation. An accompanying technical reference documents every transformation applied to the data, so that researchers can report their methods accurately.

GECKO draws a deliberate line between what it automates and what it leaves to the researcher. Preprocessing is automated: sentinel values are detected and replaced, and small gaps (50 frames by default) are interpolated linearly without prompting. Where the choice is scientifically consequential, control returns to the user — larger gaps trigger an interactive preview offering linear interpolation, saccadic (sigmoid) interpolation, or leaving the gap as `NaN`, with each decision cached per trial and per channel so it is never silently repeated. Gaze event labeling, by contrast, is entirely manual. GECKO presents the computed kinematics and event markers (for example `TARGET_ON`) and lets the user place, erase, and adjust event intervals frame by frame, but proposes no labels of its own. The aim at this stage is to establish ground truth, so transparency and correctability were judged more valuable than throughput.

Labeling a full dataset is a long process, so GECKO is built to be closed and reopened without losing work. Trial quality marks, free-text notes, export selections, label order, and per-file session state are all written to disk, so reopening a `.kinarm` file restores the session exactly where it was left, and trials can be labeled back-to-back without returning to the main window. Per-frame gaze events are written as single-digit integer codes (1 = saccade, 2 = pursuit, 3 = fixation, 9 = bad trial, 0 = unlabeled/other) in a CSV file for human-readable inspection and in a compressed NPZ archive for storage and downstream machine-learning ingestion. Channel data is kept as raw as the researcher's interpolation choices dictate, so data-preparation decisions can be made at the analysis stage.

DRAFT

Research impact statement

GECKO is the gaze-processing pipeline of the Sensorimotor Neuroscience and Learning Laboratory at The Pennsylvania State University, where it has been used to label more than 5,000 trials from 50 participants across three Kinarm studies of reaching and moving-target interception. Gaze data processed with GECKO supported work presented at the 2026 meeting of the Society for the Neural Control of Movement (Muttee et al., 2026b) and two studies presented at Neuroscience 2026, the annual meeting of the Society for Neuroscience (Gwon et al., 2026; Muttee et al., 2026a): one comparing feedforward planning and online correction during reaching and interception, and one examining how gaze position modulates online reach corrections during standing.

Two other laboratories that pair eye tracking with Kinarm robots use GECKO: the Brain and Action Laboratory at the University of Georgia (D. Barany) and the Sensorimotor Control and Robotic Rehabilitation Research Laboratory at the University of Delaware (J. Semrau). Both groups have reported issues and requested features through the public repository, including a trial review flag (#27 & 32) and macOS support (#33), which were added in later releases.

GECKO is released under the MIT license with a user manual, a technical reference documenting every transformation, a demonstration dataset with a video walkthrough, automated tests, and prebuilt Windows and macOS executables.

AI usage disclosure

Generative AI (Anthropic's Claude, various model versions, mid-2025 to present) was used as a programming assistant in the later stages of GECKO's development, accelerating code implementation, debugging, and initial drafts of documentation and manuscript text for the GUI application built on top of an initial human-developed scientific pipeline and command-line prototype. It served as an API reference and project-aware debugging aid, particularly for `numpy`, `scipy`, `pandas`, and `matplotlib` usage. All design decisions, edge-case testing, and validation were performed by the human developer, with algorithmically critical implementations verified against Singh et al. (2016) and laboratory reference implementations, and all AI-assisted outputs reviewed and approved by the authors of this manuscript.

Acknowledgements

This work was supported by the National Science Foundation under Grant No. 2444649. The funder had no role in the design of the software, the analysis approach, or the preparation of this manuscript. We thank Kinarm (BKIN Technologies, Kingston, ON, Canada) for constructive feedback during the development of GECKO.

References

Andersson, R., Larsson, L., Holmqvist, K., Stridh, M., & Nyström, M. (2017). One algorithm to rule them all? An evaluation and discussion of ten eye movement event-detection algorithms. *Behavior Research Methods*, *49*(2), 616–637. https://doi.org/10.3758/s13428-016-0738-9

Dar, A. H., Wagner, A. S., & Hanke, M. (2021). REMoDNaV: Robust eye-movement classification for dynamic stimulation. *Behavior Research Methods*, *53*(1), 399–414. https://doi.org/10.3758/s13428-020-01428-x

Gwon, D., Muttee, A., Costella, E., Liu, X., & Singh, T. (2026). Foveal versus lower-field viewing modulates the magnitude of online reach corrections during standing. *Neuroscience 2026, Annual Meeting of the Society for Neuroscience*. https://www.abstractsonline.com/pp8/#!/21503/presentation/27378

DRAFT

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hessels, R. S., Niehorster, D. C., Kemner, C., & Hooge, I. T. C. (2017). Noise-robust fixation detection in eye movement data: Identification by two-means clustering (I2MC). *Behavior Research Methods*, *49*(5), 1802–1823. https://doi.org/10.3758/s13428-016-0822-1

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Muttee, A., Amsaraj, R., Friedman, J., & Singh, T. (2026a). From reaching to intercepting: Target speed shifts the balance between feedforward motor planning and online correction. *Neuroscience 2026, Annual Meeting of the Society for Neuroscience*. https://www.abstractsonline.com/pp8/#!/21503/presentation/27377

Muttee, A., Amsaraj, R., Friedman, J., & Singh, T. (2026b). Same arm, different plan? Comparing motor planning for static reaches and moving-target interception. *Annual Meeting of the Society for the Neural Control of Movement (NCM)*. https://ncm-society.org/program/

Nyström, M., & Holmqvist, K. (2010). An adaptive algorithm for fixation, saccade, and glissade detection in eyetracking data. *Behavior Research Methods*, *42*(1), 188–204. https://doi.org/10.3758/BRM.42.1.188

Scott, S. H. (1999). Apparatus for measuring and perturbing shoulder and elbow joint positions and torques during reaching. *Journal of Neuroscience Methods*, *89*(2), 119–127. https://doi.org/10.1016/S0165-0270(99)00053-9

Singh, T., Perry, C. M., & Herter, T. M. (2016). A geometric method for computing ocular kinematics and classifying gaze events using monocular remote eye tracking in a robotic environment. *Journal of NeuroEngineering and Rehabilitation*, *13*(1), 10. https://doi.org/10.1186/s12984-015-0107-4

Startsev, M., & Zemblys, R. (2023). Evaluating eye movement event detection: A review of the state of the art. *Behavior Research Methods*, *55*(4), 1653–1714. https://doi.org/10.3758/s13428-021-01763-7

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

DRAFT